Inferno — Plain-English Guide

Read this once before you present. It explains, in ordinary words, what the project is, what every term on the slides means, and why we made each choice. Every number here matches the slides and the report exactly.

1. The one-minute summary

When an AI model writes a reply, it produces the answer one word (technically, one token) at a time. To produce even a single word, it must read its entire “brain” — billions of numbers called weights — from the GPU's memory. That reading is the slow part, not the thinking. Our project asks one question: can we make this faster by (a) hand-writing a smarter GPU program that reads less data, and (b) cleverly choosing which running copy of the model should answer each incoming request? We built both, tested them very carefully with repeated, honest measurements, and reported plainly what worked and what did not.

2. What we have done, and what is left

Done so far:

- Wrote and tested our own fast GPU program (a kernel) — it works and runs close to the fastest speed physically possible on our GPU.
- Plugged it into a real, production AI-serving system (vLLM) and measured real speed with real generated text.
- Built a “smart router” idea (adaptive routing) and fairly tested it against simple fixed choices, over 96 separate timed experiments.
- Built a message-passing router (MPI) so several running copies of the model can be coordinated.
- Built a live, interactive dashboard and a full written report — every number in them is generated automatically from real recorded data, never typed by hand.

Left to do, by the end of November:

- Test everything on multiple, separate GPUs. Right now everything runs squeezed onto ONE 8 GB GPU, which changed our results (explained in Section 5).
- Use a special GPU profiling tool (Nsight Compute) to directly confirm, with hard counters, exactly why each step of our program got faster.
- Extend our kernel so it stays fast even when many users ask questions at the same time, not just one.

3. The basic idea, as a story

Picture the AI model as a huge library. Every time it must answer, it has to walk past every single shelf — every weight/number — once, even just to produce one word. This walk is why it is slow, not the thinking. There are two ways to speed up the walk: (1) shrink the books — shrink the numbers themselves — so the walk covers less data; this is called quantization. (2) Hire more librarians — run several copies of the model — and have a receptionist decide who answers which question; this is adaptive routing. Our project builds and tests both ideas.

4. Every term on the slides, explained simply

This section is written so you can jump to any word you are unsure of. Read top to bottom once before presenting; use it as a lookup during the viva.

LLM (Large Language Model) — the AI model itself — like a mini ChatGPT. It is made of billions of numbers (weights) that decide what word comes next.

Token — a small piece of text, roughly a word or part of one. The model produces text one token at a time; that is why speed is measured in “tokens per second.”

GPU — a chip built to do many simple calculations (like multiplications) at the exact same time. It is why AI is fast at all — but it has a strict speed limit on how fast it can read data from its own memory, and that limit is what our project is really fighting.

Qwen2.5-1.5B — the actual AI model we used for every experiment. “1.5B” means 1.5 billion numbers (parameters). We chose a model this size because our GPU has only 8 GB of memory — a bigger, 8-billion-parameter model simply would not fit.

FP16 (16-bit floating point) — the normal, full-precision way to store each weight (2 bytes per number). Accurate, but takes more memory to store and more time to read.

INT4 / Quantization — instead of storing each weight at full precision, we squeeze it down to just 4 bits (about a quarter of the space). The model becomes about 4× smaller, so the GPU has to read about 4× less data — and since reading data is the bottleneck, this directly makes generation faster. The cost is a small, usually acceptable, drop in numeric accuracy.

AWQ-Int4 — one specific, well-known recipe for doing that 4-bit squeezing intelligently — it looks at which weights matter most before shrinking them, so accuracy barely drops. We used the official AWQ-Int4 release of our model rather than inventing our own quantization.

GPTQ — another popular 4-bit quantization recipe, similar in spirit to AWQ. We mention it in our literature review as related work; our own experiments use AWQ-Int4 checkpoints.

CUDA — NVIDIA's programming toolkit for writing your own custom programs that run directly on the GPU. We used CUDA to hand-write our own version of the “read the 4-bit weights and do the math” step, built specifically for our compressed format.

Kernel (in GPU programming) — just means one specific small GPU program. Our “fused int4 kernel” is the program we wrote that reads the compressed weights and multiplies them, all in a single, efficient step, without ever re-expanding them back to full size in memory.

cuBLAS — NVIDIA's own ready-made, already highly optimized library for standard math like matrix multiplication, run at full precision (FP16). We use it purely as our fair-comparison baseline — “how fast is the normal, off-the-shelf way?” — so we can prove our own kernel is genuinely faster, not just different.

Marlin — not something we built — it is an existing, extremely well-optimized 4-bit kernel that ships inside vLLM, written by other researchers. It is our main real-world “competitor” for judging how good our own hand-written kernel is.

Optimization ladder (v0 → v6) — we did not write our fast kernel in one attempt. We built it in seven small versions, each fixing exactly one inefficiency in the version before it (for example: v0’s memory access was disorganized, so v1 fixed that; v2 introduced a

hidden memory “traffic jam,” so v3 removed it; and so on). Measuring every step means we can prove each idea actually helped — and honestly report the couple of ideas that did not.

vLLM — a popular, production-quality, ready-made system (built by other people) for actually serving an LLM to many chat users efficiently. We use it as our serving foundation and plug our own kernel into it, so we can test our kernel inside a real, complete system rather than in an artificial isolated test.

Why vLLM, specifically? — because it is the most widely used, well-tested, open-source serving engine available. Comparing against it means comparing against a serious real-world system, not something we made up ourselves. It already solves the hard parts (handling many users, managing conversation memory), so we can focus on the one piece we are contributing: the kernel.

Parallelization — the 3 “levels” — “parallel” means doing many things at once instead of one after another. We use three different tools at three different scales:

- Level 1 — CUDA, inside the GPU: while computing one single answer, thousands of tiny multiplications happen at the same instant inside the chip. This is what our kernel controls.
- Level 2 — vLLM, across users on the same GPU: one GPU can work on several different users’ requests at the same time (called batching) instead of finishing one person fully before starting the next.
- Level 3 — MPI, across separate replicas/machines: if you run several full copies of the system (on different GPUs), you need a router to decide which copy handles each new request.

MPI (Message Passing Interface) — a long-standing, standard toolkit from the supercomputing world that lets separate programs — possibly on different physical machines — send short messages to each other and coordinate. We use it to build a router: one process decides which model copy should handle each new request and sends it a short “handle this one” message.

Adaptive routing / adaptive controller — our own “smart receptionist” idea: instead of always sending every request to the same copy, it watches how busy each copy is and how long each request looks, and decides in real time which copy should get the next request, aiming to keep everyone’s wait time low.

Fixed policy — the deliberately “boring” comparison: always send every request to the same one copy, with no cleverness at all. We compare our smart router against these simple, fixed choices to check whether being clever actually pays off.

p99 latency — the wait time experienced by 99 out of 100 users, or better — i.e. how slow it is for almost everyone, including the unlucky ones, not just the average person. Systems can look fine on average while some real users wait a very long time; p99 catches that.

Why did the “smart” adaptive router lose? — because we only have one physical GPU, and both copies of the model live on that same GPU. When the second copy is also made to do work, the two copies must take turns using the same GPU hardware (“time-slicing”, like two teachers sharing one whiteboard) — and that turn-taking is slower than just using one copy properly. On genuinely separate GPUs this problem should not exist — testing that is exactly our November multi-GPU plan.

Paired experiments / statistics / p-value — whenever we say “X was worse than Y,” we do not just look at one run and guess. We repeat every experiment 8 times with the exact same simulated traffic for every method being compared (a completely fair race), then run standard statistical tests to check: is this difference real, or could it just be random luck? We only report something as a finding once the maths says it is very unlikely to be luck.

Dashboard — a live webpage we built ourselves that shows all our results as interactive graphs — kernel speed, the routing experiment, and more. It is proof that this is a real, working, tested system, not just claims on slides: every number on it is generated automatically from our raw recorded log files.

5. Our results, in plain words

- Our kernel is fast. It is about 3.7× faster than the normal (FP16) way of doing the same calculation, and uses about 95% of the absolute maximum speed the GPU's memory can physically deliver — close to the physical limit of this hardware.
- Inside the real system, we tied the professional kernel. Plugged into vLLM with one user talking to it, our kernel reached 144 tokens/second — about 1.01× Marlin's speed, i.e. a tie, not a win. Building something from scratch that matches a professional, already-optimized library is itself a real result. With several users at once, Marlin pulls ahead, because it uses a more advanced part of the GPU (Tensor Cores) that our kernel does not use yet.
- Our “smart routing” idea did not win — and we know exactly why. Across 96 fair, repeated experiments, the adaptive router was 112% to 204% worse on tail latency than simply picking one fixed setup. We traced this to the two copies fighting over one shared GPU. This is an honest negative result, but a useful one: we know the cause, and we have a specific next experiment planned (separate GPUs) to properly test the original idea.
- Our MPI router itself works and is fast. Sending a message between two copies takes only a couple of millionths of a second — it is not the bottleneck; the shared GPU is.

6. How we actually ran the project, step by step

- We generate a stream of “pretend” chat requests arriving over time (sometimes steady, sometimes bursty) — a realistic simulation of real traffic.
- We send each request at exactly its scheduled time, whether or not the system has finished earlier requests — so waiting-in-line delays are honestly captured, not hidden.
- A router (fixed, adaptive, or our MPI version) decides which copy of the model handles each request.
- That copy (running inside vLLM, using FP16, INT8, or our special INT4 kernel) actually generates the reply.
- We record everything: time to the first word, time for the whole reply, and how much electricity the GPU used.
- We repeat every experiment 8 times with identical simulated traffic each time, then use statistics to check whether any difference we see is real.
- Every chart and number in our slides, report, and dashboard is generated automatically by a program reading these raw recorded files — nobody types a result in by hand.

7. Rapid-fire viva answers

What are you trying to do, in one line?

Make an AI model reply faster, by hand-writing an optimized GPU program and testing smart ways to share it among multiple requests.

Why is this a High-Performance-Computing project?

It needs thousands of simultaneous calculations (GPU/CUDA), multiple machines coordinating (MPI), and careful, statistically sound measurement of all of it.

What exactly have you achieved?

A verified, near-hardware-limit CUDA kernel; it is integrated into a real system (vLLM) and ties a professional kernel at one user; and a full, honest, statistically tested answer on whether smart routing helps (it did not, on one GPU, and we know why).

What is left before the final evaluation?

Testing on multiple separate GPUs (the real test of the routing idea), profiling with Nsight to directly confirm why the kernel is fast, and extending the kernel to stay fast with many simultaneous users.

Why did you pick such a small model (1.5B parameters)?

Our GPU has only 8 GB of memory. A bigger, 8-billion-parameter model would not fit at all; Qwen2.5-1.5B has official versions in every precision we needed to compare.

What is the single biggest lesson from this project?

Reading memory, not doing arithmetic, is what limits AI inference speed at small scale — and sharing one GPU between two things you want to compare fairly is much harder than it looks.

Is your kernel better than everything else?

No, and we do not claim that. It is a tie with Marlin at one user, and it loses with many simultaneous users. We say this plainly because it is true and it is still a meaningful result.

Did anything go wrong during the project?

Yes, and we kept it: an early measurement let a GPU cache inflate our bandwidth numbers by about 6%, and one library comparison was mismeasured. Both were caught by our own checks, fixed, and every affected result was re-run.

This guide is for your own preparation and is not part of the submitted report or slides. If a question comes up that is not covered here, it is safer to say “we have not measured that yet, and it is planned for November” than to guess.